Exercise 4 - report

Description of software.

This application core functionality is based on deepspeech module. In the application I was able to download it to function offline using pip command. For the word recognition to work on selected language, when initializing new language model, trained model file is required. That step is necessary for each language. In the future, we could change those files with models that have more data, what could improve the results of accurately recognizing words from speech.

Pipeline of this software is:

- Create language specific models from the provided models files
- Get audio files and transcriptions
- Apply noise reduction to files, save filtered files (also add silence for Italian filtered files)
- Get text from filtered audio files using deepspeech
- Calculate WER for each audio file -> text recognition result
- Show data, how text from transcripts compares to extracted text from audio, and WER avg for each language
- Create final output, as a table with language, audio file, and WER for that audio file

In software I tried to maintain modular approach of functionality, in case there will be more languages added in the future, or more testing files etc. Created function called speechRecognition that takes two parameters language model and file name and returns text that deepspeech module extracted from the audio. This allows to use this function with various languages models and files.

Since the models are loaded and function that takes language model and files return text, it's time to use that function on the files given, my recorded files, and load transcripts of what text those audio files contain. I created lists for all files names and same index related list of text transcriptions.

(Solution applied for noisy environment)

Before running function that uses speech recognition, this software first reduce noise from the background. To accomplish that application uses noisereduce module, that is also working offline. This part of software takes all the audio files, applies noise reduction with parameters that I tested different values using method of tries and errors trying to get the best results for that sample of noisy environment. Then load those files in new folder of filtered files, later on software will use those filtered files. I also added silence in front of filtered files in Italian language that greatly improved WER.

Application creates list that catches the results of speechRecognition function on all the audio files, that is done for each language.

Software has list of text results from speech recognition, as well as list of text transcriptions. Now it is time to compare those two lists elements using WER function. Store the results of those comparisons in another list for each language.

Before getting to final output, I decided that software should actually show in well organized form of table, what the transcriptions words were for each file, and what text did speech recognition function actually generated. Also in the last column put WAR values for each sentence. That allowed to really analyze how recognition of each sentence worked in my application.

Final step of software is creating the final output table that contains Language column, File column, and WAR column. Each column is filled with corresponding data.

Analysis of application results based on audio files tested.

Analyze data from software. By getting rid of some noise from the background on all files, and adding 0.5 s silence in front of audio files in Italian, application was able to get following results:

- English average WER of 8%
- Spanish average WER of 30%
- Italian average WER of 16%

English	Transcriptions	Result	WER
1my_where_terminal.wav	where is terminal five	where is terminal five	0
2my_how_get_there.wav	how do i get there	how do i get there	0
checkin.wav	where is the checkin desk	where is the checking desk	20
parents.wav	i have lost my parents	i had lost my parents	20
suitcase.wav	please i have lost my suitcase	please i have lost my suitcase	0
what_time.wav	what time is my plane	what time is my plan	20
where.wav	where are the restaurants and shops	where are the restaurants and shops	0

English avg WER:	8%

Spanish	Transcriptions	Result	WER
checkin_es.wav	donde estan los mostradores	adande estan los mostradores	25
parents_es.wav	he perdido a mis padres	he perdido a mis padres	0
suitcase_es.wav	por favor he perdido mi maleta	por favor he perdido mi maleta	0
what_time_es.wav	a que hora es mi avion	ahora es miedo	83
where_es.wav	donde estan los restaurantes y las tiendas	adande estan los restaurantes en las tierras	42

Spanish avg WER:	30%

Italian	Transcriptions	Result	WER
checkin_it.wav	dove e il bancone	dove e il pancone	25
parents_it.wav	ho perso i miei genitori	ho perso i miei genitori	0
suitcase_it.wav	per favore ho perso la mia valigia	per favore ho perso la mia valigia	0
what_time_it.wav	a che ora e il mio aereo	a che ora il mio aereo	14.2857
where_it.wav	dove sono i ristoranti e i negozi	dove sono ristoranti negozi	42.8571

Italian avg WER:	16%

Screen shot from output of Jupiter cell

The output of my work is the following table:

Language	File	WER
English	1my_where_terminal.wav	0%
English	2my_how_get_there.wav	0%
English	checkin.wav	20%
English	parents.wav	20%
English	suitcase.wav	0%
English	what_time.wav	20%
English	where.wav	0%
Spanish	checkin_es.wav	25%
Spanish	parents_es.wav	0%
Spanish	suitcase_es.wav	0%
Spanish	what_time_es.wav	83%
Spanish	where_es.wav	42%
Spanish	checkin_it.wav	25%
Spanish	parents_it.wav	0%
Italian	suitcase_it.wav	0%
Italian	what_time_it.wav	14%
Italian	where_it.wav	42%

Looks like in most cases software was quite accurate. I would maybe try to find different model file for Spanish to try to improve WAR. Since changing noise parameters didn't affect much certain audio file recognition, I think it would be beneficial to try more data samples and different models to improve the software to be even more accurate.